2025 IEEE 21st International Conference on Automation Science and Engineering (CASE)
August 17-21, 2025. Los Angeles, CA, US

2025 IEEE 21st International Conference on Automation Science and Engineering (CASE) | 979-8-3315-2246-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/CASE58245.2025.11164110

Adaptive Domain Modeling with Language Models: A Multi-Agent Approach to Task Planning

Harisankar Babu[1,2], Philipp Schillinger[1], and Tamim Asfour[2]

***Abstract*—We introduce TAPAS (Task-based Adaptation and Planning using AgentS), a multi-agent framework that integrates Large Language Models (LLMs) with symbolic planning to solve complex tasks without the need for manually defined environment models. TAPAS employs specialized LLM-based agents that collaboratively generate and adapt domain models, initial states, and goal specifications as needed using structured tool-calling mechanisms. Through this tool-based interaction, downstream agents can request modifications from upstream agents, enabling adaptation to novel attributes and constraints without manual domain redefinition. A ReAct (*Reason+Act*)-style execution agent, coupled with natural language plan translation, bridges the gap between dynamically generated plans and real-world robot capabilities. TAPAS demonstrates strong performance in benchmark planning domains and in the VirtualHome simulated real-world environment.**

I. Introduction

Adaptability to unforeseen situations and evolving requirements is crucial for effective task planning in unstructured, open-world environments. For instance, dynamic environments require robots to handle incomplete information and changing task specifications. Large Language Models (LLMs) demonstrate strong generative capabilities, enabling the parsing of natural language into structured problem representations. However, leveraging these capabilities for automated planning remains an open challenge, especially in dynamic environments requiring contextual grounding, adaptability, and reasoning under uncertainty.

Classical symbolic planning frameworks, while powerful, face notable limitations. They rely on meticulously defined domain models that require significant human expertise. This makes them rigid and difficult to adapt to novel scenarios, thus limiting their applicability in dynamic, real-world settings. Current LLM-based approaches, conversely, lack the structured reasoning capabilities of symbolic planners. This work addresses the critical need for a planning system that combines the strengths of both: the adaptability of LLMs and the rigor of symbolic planning, thereby enhancing trustworthiness.

We introduce TAPAS (Task-based Adaptation and Planning using AgentS), a multi-agent framework that bridges natural language understanding and symbolic planning for adaptive task planning. Unlike traditional approaches, TAPAS employs a set of specialized LLM-based agents, each responsible for a distinct phase of problem formulation: domain modeling, initial state generation, and goal specification. This modular design ensures structured problem representation while allowing flexible adaptation to new task constraints. Using the Unified Planning* (UP) framework, TAPAS adaptively updates domain representations through iterative refinement, automatically incorporating novel attributes such as object properties or action constraints. Figure 1 illustrates how TAPAS dynamically adapts domain models to new goal constraints.

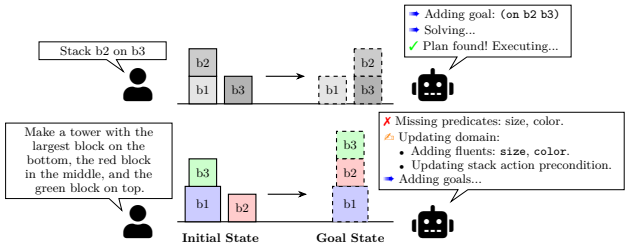


Figure 1. TAPAS dynamically adapts domain models to accommodate new goal constraints. If a goal can be represented with existing predicates, it directly generates and executes a plan. Otherwise, TAPAS detects missing predicates, updates the domain model by modifying action constraints, and integrates the new goal before solving. This allows symbolic planners to generalize beyond predefined representations while adapting to evolving task requirements.

Key contributions of TAPAS include:

1) An approach to reason over and dynamically adapt to unexpected goal specifications, including those that introduce novel attributes or constraints, without requiring manual domain redefinition. This adaptation is achieved through a modular, collaborative framework where LLM agents autonomously generate and update structured domain models, initial states, and goals.
2) A robust planning and execution pipeline that bridges the gap between dynamically generated symbolic plans and real-world robot capabilities. This is achieved through natural language translation of plans, ReAct (*Reason+Act*)-based execution to handle domain-skill discrepancies, and iterative feedback-driven validation.

By leveraging these features, TAPAS provides a scalable and adaptive approach to automated planning, addressing traditional barriers in symbolic planning. Its modular

[1]Bosch Center for Artificial Intelligence, Renningen, Germany. {harisankar.babu, philipp.schillinger}@bosch.com
[2]Karlsruhe Institute of Technology, Karlsruhe, Germany. asfour@kit.edu

*The AIPlan4EU Unified Planning Library: https://github.com/aiplan4eu/unified-planning

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 17:03:57 UTC from IEEE Xplore. Restrictions apply.

agent-based design and feedback-driven execution make it well suited for open-world environments where contextual reasoning and adaptability are essential.

II. Related Work

Effective task planning in dynamic, open-world environments requires reasoning and adaptability, which traditional methods often lack. Recently, LLMs have shown remarkable progress in reasoning, offering new avenues for these challenges. Foundational prompting techniques, such as Chain of Thought (CoT) [1, 2], established that LLMs can perform in-context reasoning. Subsequent methods like Tree of Thoughts (ToT) [3] and Graph of Thoughts (GoT) [4] further enhanced problem-solving by introducing more structured exploration of the solution space.

Building on these advancements, LLMs have been directly applied to task planning as planners. Early approaches explored translating high-level language into actionable steps [5] and grounding plans in robotic affordances [6]. To improve robustness, subsequent methods introduced iterative self-refinement through feedback loops [7, 8] and tool-interactive correction [9]. *ReAct* [10] enables interleaved reasoning and action execution, improving adaptability in dynamic tasks. Other works have focused on grounding plans in 3D scene representations [11], integrating strategic look-ahead with Monte Carlo Tree Search [12], and assessing generalization of LLM-based planners across multiple domains [13]. However, PlanBench [14] benchmarked LLM planning capabilities, highlighting gaps compared to traditional symbolic planners, motivating hybrid neuro-symbolic frameworks where LLMs assist, rather than replace, symbolic methods [15].

Consequently, hybrid models that integrate LLMs with symbolic planners have gained attention. *LLM+P* [16] combines Planning Domain Definition Language (PDDL)-based symbolic planners with LLMs to translate natural language descriptions into structured problem definitions, while *LLM-DP* [17] demonstrates how LLMs complement symbolic planners in handling uncertainty in embodied tasks. Similarly, Guan et al. [18] and *NL2Plan* [19] use multistep frameworks to generate and refine PDDL models and have been extended to multi-agent, long-horizon tasks [20]. Other approaches couple LLMs with alternative formalisms, such as Answer Set Programming (ASP) [21, 22] or signal temporal logic [23], to enhance robust reasoning for robotic planning.

TAPAS builds on these hybrid approaches, particularly those that use LLMs for problem formulation [16, 24, 25]. However, unlike systems that assume a static problem formulation, TAPAS enables iterative domain adaptation. Its agents can dynamically detect missing constraints, modify domain fluents, and refine symbolic representations. Furthermore, TAPAS integrates a procedural memory for skill transfer [26, 27] with its dynamic domain adaptation capability to continuously learn from novel problem structures.

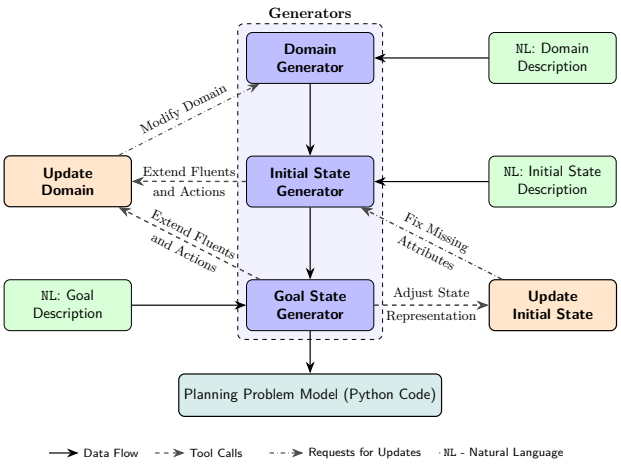


Figure 2. Multi-LLM-agent planning framework overview. The *Domain*, *Initial State*, and *Goal State* Generators iteratively refine the planning problem. When a goal requires fluents not yet represented, the system detects missing attributes and requests domain model updates, ensuring coherent problem formulation. The final output is a planning problem in Python.

III. Adaptive Domain Modeling via LLM Agents

TAPAS's ability to handle changing environments and novel task specifications centers on its adaptive domain modeling capability. This section details this core mechanism by describing the multi-agent framework and the internal architecture of the LLM agents that collaboratively construct and adapt planning domains.

A. Multi-Agent Planning Framework

TAPAS employs a multi-agent planning framework integrating the generative capabilities of LLMs with the structured reasoning of symbolic planning. The framework consists of three specialized LLM-based agents: the *Domain Generator*, *Initial State Generator*, and *Goal State Generator*. These agents operate autonomously, but collaboratively to construct well-defined planning problems. By leveraging both independent reasoning and inter-agent interactions, TAPAS effectively handles complex and dynamic environments. Each agent's system prompt emphasizes tool use and discourages assumptions.

The architecture of TAPAS's multi-agent planning framework is illustrated in Figure 2. The framework operates as follows:

- *Domain Generator*: Receives a natural language problem description and converts it into domain code, defining types, fluents, and actions in Python.
- *Initial State Generator*: Receives the domain code and a natural language description of the initial state. Generate the initial state code specifying object instances and their initial fluent values.
- *Goal State Generator*: Receives the domain code, the initial state code, and a natural language description of the goal. Defines the goal conditions, ensuring consistency with the initial state and the domain model.

Each agent validates its output both semantically and during runtime. If an agent detects errors, it iteratively

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 17:03:57 UTC from IEEE Xplore. Restrictions apply.

Agent Tools for Domain Modification:

- Tool name: `missing_or_incorrect_fluent`
  Arguments: `fluent_name`, `fluent_description`
  Request the addition or modification of a fluent.
- Tool name: `action_modification`
  Arguments: `action_name`, `change_description`
  Requests modifications to an existing action.

Agent Tools for Initial State Modification:

- Tool name: `missing_objects`
  Arguments: `object_type`, `object_description`
  Requests the addition of objects required for the goal.

Figure 3. Agent tools for modifying domain and initial state.

refines its output until a valid solution is achieved or a predefined correction limit is reached.

Collaborative Refinement and Adaptation via Tool Use: A key strength of TAPAS is its iterative refinement process, in which LLM-based agents actively detect, correct, and adapt planning problem definitions in response to evolving constraints through structured tool use. Downstream generators analyze upstream output, detecting missing attributes or ill-defined elements. When inconsistencies arise, agents engage in a structured feedback loop, invoking specific tools to request modifications, as detailed in Figure 3.

The Initial State Generator may also query the user for additional information (e.g., the color of an object) if not provided in the initial description. This tool-based interaction ensures that only downstream agents trigger changes in upstream components, maintaining a consistent and well-defined problem structure. The tools are modular, allowing for easy extension with new capabilities.

For instance, in the classic *blocksworld* domain, blocks are typically represented as objects (e.g., *b1, b2, b3*). If the user specifies a goal such as *"place the blue block on top of the red block"*, the *Goal State Generator* identifies that color attributes are absent from the initial state and domain model. It then invokes the `missing_or_incorrect_fluent` tool to request adding a `color` fluent. This, in turn, prompts the *Initial State Generator* to incorporate these attributes and the *Domain Generator* to update the domain model. This bottom-up, tool-mediated adaptation ensures consistency across all problem levels. This mechanism allows TAPAS to handle a wide range of novel constraints and attributes, not just predefined ones.

By iteratively resolving inconsistencies through structured tool use, TAPAS surpasses traditional planning approaches that rely on static problem formulations. Agents not only correct errors but also evolve problem representation in response to new constraints or goals, enabling flexible and adaptive planning in dynamic real-world environments.

B. Agent Architecture

Each agent in TAPAS's modular architecture integrates reasoning, refinement, and memory. Agents operate independently but communicate via structured tool-calls, ensuring coherent problem formulation and enabling flexible task decomposition and adaptation.

A TAPAS agent's workflow comprises three key phases: *Generation*, where the agent produces structured representations of planning problems based on input queries and available context; *Evaluation and Refinement*, where the agent assesses its output, incorporates relevant feedback, and refines the response if necessary; and *Execution and Feedback*, where the validated output is executed within the planning pipeline, with any execution feedback integrated into the reasoning process.

1) Memory Mechanisms: To support task adaptation, TAPAS primarily relies on a short-term context memory ($\mathcal{M}_{\text{context}}$), a buffer which stores recent interactions like user queries, tool calls, and agent responses. This enables multi-turn coherence and keeps the agent focused on the immediate task. As an auxiliary mechanism for long-term learning across different tasks, TAPAS also includes a procedural memory ($\mathcal{M}_{\text{procedural}}$), a long-term repository for storing generalizable corrections.

Memory Storage and Retrieval: The procedural memory is designed to learn incrementally from individual, significant events. Storage is prompted by corrections deemed to be generalizable, which are typically those resulting from explicit user feedback. Upon identifying such a correction, the agent invokes a dedicated memory storage tool. For retrieval, given a new task query q, the agent computes similarity scores between q and stored summaries $\{s_i\}$ in $\mathcal{M}_{\text{procedural}}$: $\text{score}(q, s_i) = e_q \cdot e_{s_i} / \|e_q\| \|e_{s_i}\|$, where e_q and e_{s_i} are embedding representations of the task query and stored feedback summary, respectively. High-scoring entries are integrated into the agent's reasoning process. However, this similarity-based retrieval has limitations, as it can retrieve memories that are syntactically similar but contextually irrelevant, potentially leading the agent to generate a flawed or unsolvable planning problem.

2) Self-Reflection Mechanism: To enhance response quality, TAPAS employs a self-reflection mechanism where a generator agent collaborates with an LLM critic. The critic evaluates the generator's initial response, r, providing detailed feedback and a quality score $\sigma \in [0, 1]$. Instructions for the critic to determine σ include criteria like correctness, coherence, and completeness, as well as a quality threshold τ for acceptance. The critic acts as an evaluator, analyzing the response and generating both a numerical score and textual feedback explaining any identified issues or areas for improvement.

If $\sigma \geq \tau$, the response is deemed acceptable. Otherwise, the generator incorporates the critic's feedback to refine its response. This iterative process continues until the score meets the threshold or a predefined iteration limit is reached. If the limit is reached, the system proceeds with the latest response. This represents a design trade-off between robustness and computational cost, as accepting a lower-quality response may lead to subsequent errors in downstream modules or a plan failure.

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 17:03:57 UTC from IEEE Xplore. Restrictions apply.

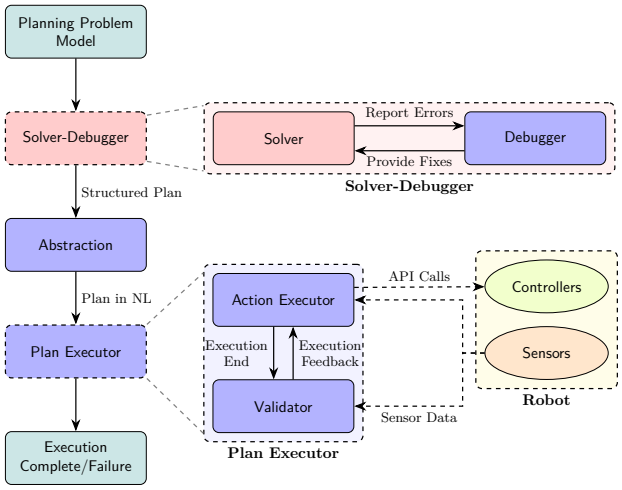


Figure 4. TAPAS's execution pipeline. The solver generates a structured plan, translated into natural language instructions and executed by the plan executor. The plan executor consists of the action executor, which performs actions, and the validator, which monitors execution and determines whether to continue or terminate based on goal fulfillment.

IV. Plan Solving and Execution

Once the planning problem is defined, the next challenge is generating and executing a plan. This section describes the components for plan generation and plan execution. Figure 4 overviews this pipeline. A key TAPAS feature is its ability to handle generated domain models, where action names and arguments may not directly correspond to available robot skills, requiring the flexible plan solving and execution approach described below.

A. Solver

The solver uses the UP framework, which supports multiple underlying planning formalisms like PDDL, to generate a sequential plan from the provided domain, initial state, and goal specifications. If semantic or modeling errors are detected, the solver invokes a Retrieval Augmented Generation (RAG)-assisted LLM debugger.

The debugger queries a vector database containing UP framework documentation. The database is constructed offline by embedding relevant UP framework documentation using a pre-trained sentence embedding model. This process provides concrete code corrections to resolve solver errors, improving robustness and helping to ensure the generated plan adheres to problem constraints.

B. Plan Abstraction

Because the domain model is generated, its actions might not have a one-to-one correspondence with the robot's available skills; they might have different names and arguments. Therefore, direct execution of the solver's output might be impossible. The purpose of plan abstraction is to bridge this semantic gap by decoupling the high-level symbolic plan from the robot's specific, low-level skill library.

To achieve this, the LLM-based plan abstraction converts the solver's structured plan into natural language instructions. This translation serves as an intermediate representation that facilitates the subsequent grounding of abstract plan steps (e.g., "place a block") to concrete skills by a dedicated execution agent, as described in the following section. This process makes the plan understandable to human operators and is also suitable for modern vision-language-action (VLA) models [28, 29]. An example of a structured plan and its translation is shown in Figure 5.

Structured Plan:
1) move(pos-0-1, pos-0-2, h0)
2) place_block(pos-0-2, pos-0-3, h0, h1)
3) remove_block(pos-0-2, pos-0-3, h0, h1)

Translated Instructions:
1) Move from position *pos-0-1* to position *pos-0-2*.
2) Place a block at position *pos-0-3* from position *pos-0-2*.
3) Remove a block at position *pos-0-3* from position *pos-0-2*.

Figure 5. A structured plan translated into natural language.

Here, internal parameters such as height (*h0*, *h1*) are omitted since they do not affect fundamental task understanding. This ensures the translated instructions are concise, user-friendly, and actionable. Despite this translation, the generated plan might not be directly executable and requires further processing by a dedicated executor.

C. Plan Executor

To perform the crucial step of aligning the abstracted natural language plan with the robot's available skills, the plan executor employs a flexible strategy within the execution environment. It consists of two LLM-based agents: the *Action Executor Agent* and the *Validator Agent*, which collaborate to ensure successful execution.

The core execution workflow is as follows:

1) The executor receives the current environment state. This can be text-based (e.g., symbolic state descriptions) or combine text and images, providing a view of the environment along with textual state descriptions.
2) The Action Executor Agent sequentially processes each step in the translated plan, treating each as a short-term sub-goal. Action execution uses a ReAct-style approach [10] to map the natural language instruction to the most appropriate available skill (tool) and its corresponding parameters.
3) The Action Executor Agent then executes the selected skill.

After the Action Executor Agent completes the action sequence or encounters an unrecoverable error, the Validator Agent reviews the execution logs and the final environment state. If the overall goal is met, the process terminates. Otherwise, the Validator provides corrective feedback to re-invoke the Action Executor with modified instructions, potentially attempting a different skill sequence. For critical failures, the Validator notifies the user instead of continuing execution.

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 17:03:57 UTC from IEEE Xplore. Restrictions apply.

TABLE I
ACCURACY OF TAPAS ON LLM+P BENCHMARK DOMAINS.

Domain Name	Generation Status	Problems Solved (%)					LLM+P Reported (%)
		GPT-4o	Claude 3.7 Sonnet	GPT-4o Mini	Mistral Large	Cohere command-r	
Barman	✓	97	**100**	10	15	25	20
Blocksworld	✓	**100**	95	70	5	**100**	90
Floortile	✓	57	**100**	0	15	0	0
Grippers	✓	**100**	**100**	50	15	**100**	**100**
Storage	✓	90	**100**	0	20	20	85
Termes	✓	95	**100**	0	15	0	20
Tyreworld	✓	80	**95**	0	0	0	90

V. RESULTS

We evaluated TAPAS on benchmark planning problems from LLM+P [16] and additional experiments designed to assess its ability to adaptively update domain representations and adapt to new constraints. The experiments focus on three key aspects: (1) domain generation and planning accuracy on standard benchmarks, (2) handling new attributes in goal specification, and (3) evaluating the framework in the *VirtualHome* simulation environment [30].

a) Experimental Setup: All experiments utilized the GPT-4o[†] model (version 2024-08-06) via the Azure OpenAI Service API. The model was used without fine-tuning, with a single generic example of the UP framework provided in the system prompt to avoid in-domain influence. A temperature of 0.0 was used unless otherwise specified, with default values for *top-p* and other parameters. To prevent infinite loops, a recursion limit of 10 was enforced, restricting the maximum number of iterative steps in the planning process.

A. Benchmark Evaluation on LLM+P Domains

TAPAS was evaluated on seven planning domains from LLM+P [16], encompassing a range of classical planning tasks. The evaluation consisted of two stages: domain generation and problem solving using the generated domain. Note that LLM+P utilized GPT-4 for its evaluation along with in-domain example.

For domain generation, TAPAS inferred types, fluents, and action models from natural language descriptions of each domain, including actions, preconditions, and effects. Downstream agents provided feedback for modifications in case of inconsistencies or missing fluents. The generated domains were then used with a planner, and the solutions were validated against ground-truth data. Context memory was cleared between runs to ensure independence and no procedural memory was used.

a) Results on Domain Generation: TAPAS successfully generated executable domain models for all seven domains. Downstream agent feedback was minimal, primarily limited to the *barman* domain, where the initial state generator identified missing fluents related to cocktail ingredients, leading to a refinement of the model.

[†]GPT-4o: https://cdn.openai.com/gpt-4o-system-card.pdf

TABLE II
IMPACT OF TEMPERATURE ON PLANNING SUCCESS RATE

Domain	Temperature		
	0.0	0.1	0.3
Barman	97	**100**	**100**
Blocksworld	**100**	**100**	95
Floortile	57	**70**	47
Grippers	**100**	**100**	97
Storage	90	**92**	82
Termes	95	**100**	97
Tyreworld	80	**85**	67
Average	**88.42**	**92.42**	**83.57**

b) Results on Planning: Each generated domain was used to solve all 20 problem instances from the corresponding benchmark dataset. To test the robustness of the downstream agents, the successfully generated domain for each benchmark was provided directly, bypassing the initial domain generation stage. The initial and goal state generators were still permitted to request modifications, which assessed their ability to work with a correct domain without introducing flawed changes.

Table I presents TAPAS's accuracy across the domains, calculated as the average percentage of correctly solved problems over two repetitions of the 20 problems. The model consistently produced correct solutions, demonstrating strong generalization across diverse planning tasks.

TAPAS demonstrated robustness against false positive domain modification requests from downstream agents. When fluents were mistakenly reported as missing, the domain generator correctly rejected unnecessary changes, maintaining the original model. Most failures occurred in the *floortile* and *tyreworld* domains. In *floortile*, the LLM misidentified directional fluents, resulting in incorrect movement constraints. In *tyreworld*, additional fluents were introduced, leading to over-specified goals. Notably, these errors were not correlated with problem complexity, such as the number of objects, but with the interpretation of domain constraints.

To investigate factors influencing TAPAS's performance, ablations were performed using different closed and open-source models and temperature settings. Model ablations were performed using Claude 3.7 Sonnet[‡], GPT-4o Mini,

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 17:03:57 UTC from IEEE Xplore. Restrictions apply.

TABLE III
ADAPTABILITY TO NOVEL GOAL CONSTRAINTS

Domain	New Constraint Type	Success Rate
Blocksworld	Color-based goal	100%
Blocksworld	Size-based goal and action	90%
Grippers	Battery level goal and action	100%
Floortile	Battery level goal and action	70%

Mistral Large[§] - 2411, and Cohere Command-R[¶] - 08-2024, across all previously generated domains from GPT-4o. Additionally, the impact of sampling temperatures between 0.0 and 0.3 was evaluated, as higher values can introduce variability in code generation [31]. Table II displays the average observed success rate for each temperature in multiple runs. Performance decreased at a temperature of 0.3, as LLM introduced extraneous fluents and deviated from system instructions. However, this behavior may be advantageous in scenarios requiring adaptive extension of domain attributes.

B. Evaluation on New Attributes in Goal Specification

To evaluate TAPAS's adaptability to novel goal constraints, we conducted experiments requiring domain modifications that would typically need manual extension. The tests involved introducing `color` and `size` attributes to *blocksworld*, and numeric `battery_level` constraints to the *grippers* and *floortile* domains. The results in four sets of five problems are summarized in Table III.

When a goal required a new descriptive attribute, such as color, TAPAS correctly inferred the need for a new fluent. The *Domain Generator* added a `color` predicate to the model, and the *Initial State Generator* then queried the user for the color of each block, ensuring a consistent and solvable problem definition.

More significantly, TAPAS demonstrated the ability to modify core action logic to satisfy functional constraints. For goals involving object size or battery consumption, the system updated action preconditions and effects. For instance, to handle a size-based constraint, TAPAS introduced a `size` fluent and altered the `stack` action. This required updating the action's precondition to enforce the new stacking rule as shown in Figure 6.

A similar process was used to add a `battery_level` fluent and modify the effects of `move` and `paint` actions to consume battery. The natural language goal prompt specified the precise cost of each action (e.g., "each move action costs 5 battery units") and the final goal condition (e.g., "robot must have at least 20 battery units left"). The agent was then expected to query the user for the initial battery levels.

The results demonstrate TAPAS's ability to autonomously refine domain models, a capability beyond traditional planners. The lower success rate in *floortile* was primarily due to the agent assuming an initial battery level instead of querying the user, highlighting the ongoing challenge of preventing agent assumptions.

New Goal: *"The goal is to move the blocks to make a tower with the largest block on the bottom and the smallest block on top. Ensure that a block can be stacked only on top of a larger block in the action."*

```
- (:requirements :strips :typing)
+ (:requirements :strips :typing
↪  :numeric-fluents)
  (:types
    block - object
  )
+ (:functions (size ?b))
  (:action stack
  :parameters (?b1 ?b2 - block)
- :precondition (and (holding ?b1) (clear
↪  ?b2))
+ :precondition (and (holding ?b1) (clear
↪  ?b2) (< (size ?b1) (size ?b2)))
```

Figure 6. Automatic modification of the generated PDDL domain (bottom) to incorporate additional requirements (top).

C. Evaluation in VirtualHome Simulation

In addition to the benchmarking study, experiments were conducted within the VirtualHome simulation. We provide this evaluation to showcase the application of the entire TAPAS framework in a complex planning and execution pipeline. VirtualHome provides a 3D household simulation where humanoid agents interact with objects to execute tasks from language instructions.

a) Experimental Setup: The task is to achieve two goals: placing `pie` on `kitchentable` and warming and placing `salmon` on the same table. TAPAS received task descriptions and initial environment states in natural language. The system generated structured plans, translated them into executable instructions, and sequentially executed each step within the simulation. The *plan executor* interacted with the environment using predefined skills, receiving first-person visual feedback and textual responses. In this simulated environment, information like object states and locations is obtained directly from the simulation state, which is then converted into the textual and visual observations provided to the executor, simulating the output of a real-world perception stack. Execution continued until all plan steps were completed, and goal satisfaction was verified.

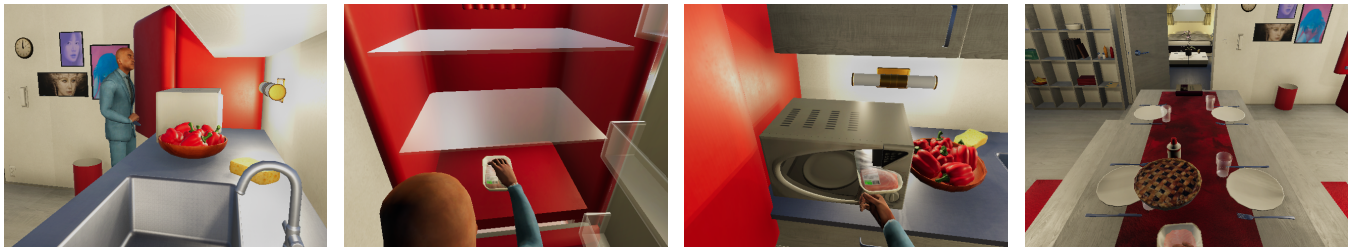
b) Results on Execution: TAPAS successfully generated and executed structured plans to achieve the specified goals. The humanoid agent interacted with the fridge, retrieved the salmon, used the microwave for heating, and placed it on the kitchen table. The system demonstrated

"Save the following to memory: for problems involving the fridge, append a goal to close the fridge, even if not explicitly stated."

Figure 7. Explicit user feedback for the procedural memory.

[‡]Claude 3.7: https://www.anthropic.com/news/claude-3-7-sonnet
[§]Mistral Large 2: https://mistral.ai/news/mistral-large-2407
[¶]Command R: https://docs.cohere.com/v2/docs/command-r

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 17:03:57 UTC from IEEE Xplore. Restrictions apply.



(a) Open fridge (b) Grab salmon (c) Heat up (d) Place salmon on kitchen table

Figure 8. Plan execution in VirtualHome: The humanoid agent sequentially interacts with objects to complete the task. (a) Opening fridge, (b) salmon retrieval, (c) heating, and (d) placement on the kitchen table.

effective translation of high-level goals into executable steps, showcasing robust task planning and execution.

To evaluate TAPAS's procedural memory, the goal state generator received an instruction mimicking user feedback as shown in Figure 7 Upon encountering a similar task after context clearance, TAPAS successfully recalled and applied the stored instruction. The planner autonomously added the goal condition: `(Not (is_open fridge_305))`. This ensured fridge closure despite its absence from the new task description. This demonstrates TAPAS's potential for long-term adaptation across tasks, though further evaluation of consistency and scalability is necessary.

c) Plan Execution in VirtualHome: Figure 8 illustrates key plan execution steps. Snapshots depict the humanoid agent retrieving, heating, and placing food items. This demonstrates TAPAS's ability to translate abstract goals into executable steps while adapting to dynamic visual feedback.

VI. Discussion

TAPAS's core contribution is its ability to autonomously generate and adaptively update domain models, initial states, and goal specifications, facilitated by collaborative LLM agents. This adaptive mechanism distinguishes it from static, predefined planning approaches, demonstrated by its successful adaptation to novel attributes in *blocksworld* and execution in VirtualHome.

The performance of TAPAS is intrinsically linked to the capabilities of the underlying LLM, as confirmed by our model ablation studies. Overall, Claude 3.7 Sonnet and GPT-4o achieved the highest performance with a clear margin over the other investigated LLMs. This underscores the importance of selecting appropriate models and investigating their generalizability.

Mitigating LLM hallucinations and errors remains crucial for robust real-world deployment. While self-reflection and refinement mechanisms address certain errors, deeper biases and complex hallucination, observed particularly with smaller models, remain a challenge. While a detailed analysis of this behavior is beyond the scope of this work, this problem could potentially be mitigated by the inclusion of in-domain examples.

In application scenarios, error detection during plan execution relies on the Validator Agent's role in identifying discrepancies between executed actions and the expected state. While the system can attempt corrective feedback loops, irrecoverable errors such as fundamental task misunderstandings currently lead to user notification.

Future research can build upon the presented framework to enhance its robustness and expand its capabilities. For instance, rigorous ablation studies could quantify the contribution of individual components to inform the design of more efficient neuro-symbolic architectures. The system's generalization could be further tested through evaluation against larger benchmarks and in less-structured, dynamic environments. Such testing would identify critical failure modes and guide improvements in planner robustness, including strategies to achieve strong performance with smaller, resource-efficient LLMs.

A key long-term objective is to enhance operational autonomy by developing sophisticated closed-loop execution cycles. Enabling the system to recover from errors by re-engaging the planner is a critical step towards reliable real-world deployment.

VII. Conclusion

TAPAS introduces a significant advancement in adaptable planning systems for open-world environments. By dynamically adapting domain models and effectively handling novel constraints, TAPAS achieved an 88.42% success rate across diverse benchmark planning domains and demonstrated successful execution in a simulated real-world setting. These results highlight the potential of integrating LLM-driven adaptability with the structured reasoning of symbolic planning.

Future research will focus on incorporating hierarchical and temporal reasoning, expanding TAPAS's capabilities to a broader range of environments, and integrating human-in-the-loop feedback. TAPAS provides a robust foundation for developing adaptable and trustworthy planning agents capable of navigating the complexities of real-world tasks.

The code will be available shortly at https://sites.google.com/view/adaptive-llm-planning.

ACKNOWLEDGMENT

We thank Patrick Kesper and Dragan Milchevski for support in setting up endpoints on AWS and Azure, respectively.

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 17:03:57 UTC from IEEE Xplore. Restrictions apply.

References

[1] J. Wei, X. Wang, D. Schuurmans, M. Bosma, b. ichter, F. Xia *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24 824–24 837.

[2] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 22 199–22 213.

[3] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 11 809–11 822.

[4] M. Besta, N. Blach, A. Kubicek, R. Gerstenberger, M. Podstawski, L. Gianinazzi *et al.*, "Graph of thoughts: Solving elaborate problems with large language models," *Proc. of the AAAI Conf. on Artificial Intelligence*, vol. 38, no. 16, pp. 17 682–17 690, Mar. 2024.

[5] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, "Language models as zero-shot planners: Extracting actionable knowledge for embodied agents," in *Proc. of the 39th Int. Conf. on Machine Learning*, vol. 162, 17–23 Jul 2022, pp. 9118–9147.

[6] B. Ichter, A. Brohan, Y. Chebotar, C. Finn, K. Hausman, A. Herzog *et al.*, "Do as i can, not as i say: Grounding language in robotic affordances," in *Proc. of The 6th Conf. on Robot Learning*, vol. 205, 14–18 Dec 2023, pp. 287–318.

[7] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence *et al.*, "Inner monologue: Embodied reasoning through planning with language models," in *Proc. of The 6th Conf. on Robot Learning*, vol. 205, 14–18 Dec 2023, pp. 1769–1782.

[8] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 8634–8652.

[9] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, N. Duan, and W. Chen, "Critic: Large language models can self-correct with tool-interactive critiquing," *arXiv preprint arXiv:2305.11738*, 2023.

[10] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "React: Synergizing reasoning and acting in language models," *arXiv preprint arXiv:2210.03629*, 2022.

[11] K. Rana, J. Haviland, S. Garg, J. Abou-Chakra, I. Reid, and N. Suenderhauf, "Sayplan: Grounding large language models using 3d scene graphs for scalable robot task planning," in *Proc. of The 7th Conf. on Robot Learning*, vol. 229, 06–09 Nov 2023, pp. 23–72.

[12] S. Hao, Y. Gu, H. Ma, J. Hong, Z. Wang, D. Wang, and Z. Hu, "Reasoning with language model is planning with world model," in *Proc. of the Conf. on Empirical Methods in Natural Language Processing*, 2023, pp. 8154–8173.

[13] T. Silver, S. Dan, K. Srinivas, J. B. Tenenbaum, L. Kaelbling, and M. Katz, "Generalized planning in pddl domains with pretrained large language models," *Proc. of the AAAI Conf. on Artificial Intelligence*, vol. 38, no. 18, pp. 20 256–20 264, Mar. 2024.

[14] K. Valmeekam, M. Marquez, A. Olmo, S. Sreedharan, and S. Kambhampati, "Planbench: An extensible benchmark for evaluating large language models on planning and reasoning about change," *Advances in Neural Information Processing Systems*, vol. 36, pp. 38 975–38 987, 2023.

[15] S. Kambhampati, K. Valmeekam, L. Guan, M. Verma, K. Stechly, S. Bhambri *et al.*, "Llms can't plan, but can help planning in llm-modulo frameworks," in *Forty-first Int. Conf. on Machine Learning*, 2024.

[16] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone, "Llm+ p: Empowering large language models with optimal planning proficiency," *arXiv preprint arXiv:2304.11477*, 2023.

[17] G. Dagan, F. Keller, and A. Lascarides, "Dynamic planning with a llm," *arXiv preprint arXiv:2308.06391*, 2023.

[18] L. Guan, K. Valmeekam, S. Sreedharan, and S. Kambhampati, "Leveraging pre-trained large language models to construct and utilize world models for model-based task planning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 79 081–79 094.

[19] E. Gestrin, M. Kuhlmann, and J. Seipp, "Nl2plan: Robust llm-driven planning from minimal text descriptions," *CoRR*, 2024.

[20] X. Zhang, H. Qin, F. Wang, Y. Dong, and J. Li, "Lamma-p: Generalizable multi-agent long-horizon task allocation and planning with lm-driven pddl planner," *arXiv preprint arXiv:2409.20560*, 2024.

[21] X. Lin, Y. Wu, H. Yang, Y. Zhang, Y. Zhang, and J. Ji, "Clmasp: Coupling large language models with answer set programming for robotic task planning," *arXiv preprint arXiv:2406.03367*, 2024.

[22] Z. Yang, A. Ishay, and J. Lee, "Coupling large language models with logic programming for robust and general reasoning from text," in *Findings of the Association for Computational Linguistics: ACL*, 2023, pp. 5186–5219.

[23] Y. Chen, J. Arkin, C. Dawson, Y. Zhang, N. Roy, and C. Fan, "Autotamp: Autoregressive task and motion planning with llms as translators and checkers," in *IEEE Int. Conf. on Robotics and Automation*, 2024, pp. 6695–6702.

[24] T. Birr, C. Pohl, A. Younes, and T. Asfour, "AutoGPT+P: Affordance-based Task Planning using Large Language Models," in *Proc. of Robotics: Science and Systems*, Delft, Netherlands, July 2024.

[25] Y. Liu, L. Palmieri, S. Koch, I. Georgievski, and M. Aiello, "Delta: Decomposed efficient long-term robot task planning using large language models," *arXiv preprint arXiv:2404.03275*, 2024.

[26] C. Pohl, F. Reister, F. Peller-Konrad, and T. Asfour, "Makeable: Memory-centered and affordance-based task execution framework for transferable mobile manipulation skills," in *IEEE/RSJ Int. Conf. on Intelligent Robots and Systems*, 2024, pp. 3674–3681.

[27] H. Ali, P. Allgeuer, C. Mazzola, G. Belgiovine, B. C. Kaplan, L. Gajdošech, and S. Wermter, "Robots can multitask too: Integrating a memory architecture and llms for enhanced cross-task robot action generation," in *IEEE-RAS 23rd Int. Conf. on Humanoid Robots*, 2024, pp. 811–818.

[28] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair *et al.*, "Openvla: An open-source vision-language-action model," *arXiv preprint arXiv:2406.09246*, 2024.

[29] A. O'Neill, A. Rehman, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee *et al.*, "Open x-embodiment: Robotic learning datasets and rt-x models," in *IEEE Int. Conf. on Robotics and Automation*, 2024, pp. 6892–6903.

[30] X. Puig, K. Ra, M. Boben, J. Li, T. Wang, S. Fidler, and A. Torralba, "Virtualhome: Simulating household activities via programs," in *Proc. of the IEEE Conf. on Computer Vision and Pattern Recognition*, 2018, pp. 8494–8502.

[31] M. Renze, "The effect of sampling temperature on problem solving in large language models," in *Findings of the Association for Computational Linguistics: EMNLP*, Miami, Florida, USA, Nov. 2024, pp. 7346–7356.

Authorized licensed use limited to: VIT-Amaravathi campus. Downloaded on October 31,2026 at 17:03:57 UTC from IEEE Xplore. Restrictions apply.